Networked Robot Teleoperation — Project Overview

Oct 16, 2026 · @Yash Aggarwal

1. What the project is

Title: Networked Teleoperation of a Robot through Shell Commands

Problem: Control a remote robot using commands typed in a terminal and sent over a TCP (Transmission Control Protocol) network connection. Add a heartbeat and an emergency stop for safety, and study how network delay affects closed-loop control.

Real-world use: In nuclear, chemical and other hazardous plants, people cannot safely stand near the equipment. Operators sit in a safe control room and drive robots remotely. This project is a simulated version of that setup.

Hardware: None. The robot lives inside the CoppeliaSim simulator, and the whole system runs on a single laptop.

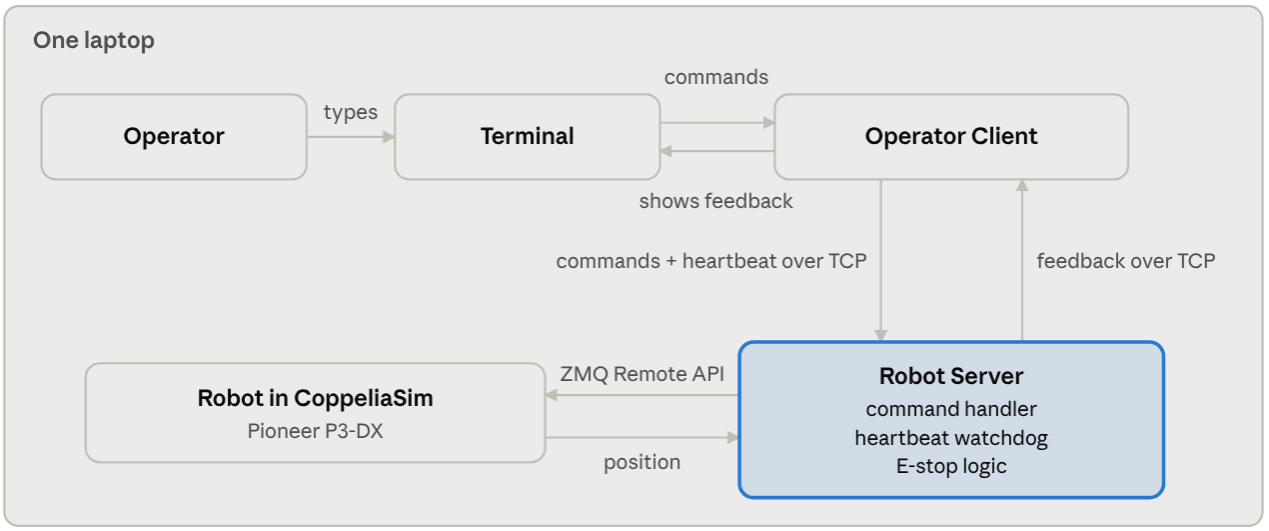
2. How the system works

Everything runs on **one laptop** as two separate programs that talk to each other over a TCP connection. The operator types a command such as "forward 0.5" into a terminal. The first program, the **Operator Client**, reads that command and sends it over TCP.

The second program, the **Robot Server**, receives the command and moves the robot inside CoppeliaSim. It also reads the robot's position from the simulator and sends it back to the Operator Client as feedback, which is shown to the operator on the terminal.

This creates a loop: command goes out, robot moves, feedback comes back, operator decides the next command. That loop is called **closed-loop control**.

Commands go down to the robot over TCP and feedback comes back up



System on one laptop · commands, heartbeat and feedback over TCP

Even though both programs share one laptop, they communicate only through the TCP connection, exactly as they would if they were on separate machines. This keeps the setup realistic for a remote-control system.

3. Components

Component	What it does
Operator	The human who types commands and watches feedback.
Terminal	The command-line window where commands are typed and feedback is displayed.
Operator Client	Program that reads typed commands, sends them over TCP, sends heartbeats, receives and displays feedback, and records timings.
TCP connection	A reliable link between the two programs that carries commands, heartbeats and feedback.
Robot Server	Program that receives commands, controls the robot, watches the heartbeat, handles the emergency stop and sends feedback.
ZMQ Remote API (ZeroMQ)	The bridge that lets the Robot Server control the robot inside CoppeliaSim.

Component	What it does
CoppeliaSim	The 3D simulator with physics where the robot lives.
Pioneer P3-DX	A ready-made two-wheeled mobile robot, driven by setting its left and right wheel speeds.

4. Why these tools

- **CoppeliaSim:** free, works on Windows, Mac and Linux, includes ready-made robots, and is easy to control from Python. Alternatives like Gazebo need ROS (Robot Operating System) and Linux, which take longer to learn.
- **Pioneer P3-DX:** built into CoppeliaSim and simple to drive, since it only needs two wheel speeds. Its top speed is about 1.2 m/s, so realistic commands are around 0.3 to 0.5 m/s.
- **Python:** CoppeliaSim provides an official Python library, and networking is straightforward in Python.
- **TCP:** it guarantees that messages arrive in the right order and without loss, which matters for safety commands. It is also built into Python, so nothing extra is needed.

5. Commands the operator can use

Command	What happens
forward / back with a speed	Robot drives straight at that speed
left / right with an angle	Robot turns in place
stop	Robot stops normally
goto with an x and y position	Robot drives to that point using position feedback
status	Shows the robot's position and current state
estop	Emergency stop: robot stops and locks
reset	Unlocks the robot after an emergency stop

Every message sent between the two programs carries a **timestamp**. The receiver compares it with its own clock to calculate how late the message arrived.

6. Safety features

Heartbeat (automatic)

The heartbeat protects against the **connection** failing.

- The Operator Client sends a small "I am alive" message to the Robot Server every **0.5 seconds**, continuously in the background.
- The Robot Server runs a **watchdog** that keeps checking when the last heartbeat arrived.
- If no heartbeat arrives for **1.5 seconds** (about three missed beats), the Robot Server assumes the connection is lost and **stops the robot by itself**.
- When heartbeats return, the robot stays still until the operator sends a **new** command. It never resumes the old command on its own, because the situation may have changed.
- The timeout is 1.5 seconds so that a single late message does not cause a false stop, while a real disconnection is still caught quickly.

E-stop (manual)

The E-stop (Emergency Stop) protects against a **dangerous situation** the operator sees.

- The operator types "estop". The Robot Server handles it immediately, ahead of any waiting commands.
- The robot stops and becomes **locked (latched)**. While locked, it ignores all movement commands.
- Only a "reset" command unlocks it, and the robot still waits for a new command before moving.
- E-stop is **not a shutdown**. The Robot Server keeps running, the connection stays open, and feedback keeps flowing to the operator.
- The lock exists because a delayed movement command that was already travelling over the network could otherwise restart the robot right after the stop.

Robot states

The Robot Server always keeps the robot in one of four states:

State	Meaning	Accepts movement commands?
IDLE	Connected and not moving	Yes
MOVING	Carrying out a command	Yes

State	Meaning	Accepts movement commands?
COMM_LOST	Heartbeat timed out; robot stopped	No, until heartbeats return and a new command arrives
ESTOP_LATCHED	Emergency stop active; robot locked	No, until reset

E-stop has the highest priority and overrides every other state.

	Heartbeat stop	E-stop
Triggered by	Robot Server automatically	Operator manually
Cause	Connection lost	Danger seen by operator
Effect	Robot stops	Robot stops and locks
To move again	Heartbeats return + new command	Reset + new command

7. Network delay study

In closed-loop control, the operator acts on feedback. With network delay, the feedback is old and commands arrive late, similar to driving a car while seeing the road one second behind.

How delay is created: the programs deliberately hold each message for a fixed time before using it. This works on any operating system and gives exact, repeatable delay values.

Delay values tested: 0, 100, 300, 500 and 1000 ms (milliseconds).

Tasks used:

- Driving to a target point
- Following a square path

What is measured:

- Final position error: how far the robot ends from the target
- Overshoot: how far it goes past the target
- Oscillation: whether it wobbles back and forth
- Task completion time
- E-stop reaction time: how long the robot takes to stop after the command
- False heartbeat stops at high delay

Results produced: a graph of delay against position error, the robot's path drawn at each delay value, and a results table.

Expected outcome: small delays cause little harm. As delay grows, the robot overshoots, wobbles, takes longer, and responds to E-stop more slowly. This shows why real teleoperation systems need ways to handle delay.